CSCC43 Project:

Database Design Project

(MyTix)

Tooba Jalal: 1010493789,
Rabiah Islam: 1011376912

1. Project Description

1.1. Purpose

Description:
This database project supports the operations of an event ticketing platform. The purpose of this application is to manage two different types of users, including customers and organizers, where organizers create and manage events with multiple performances, manage venue seating, and track the number of tickets sold, cancellations, reviews and resales. On the other hand, customers are provided with a platform to purchase tickets for different performances of events such as concerts, comedy shows, etc. The system supports multiple customer and organizer operations, and the database is designed to store and manage information related to users, events, venues, performances, tickets and purchases.

1.2. Conceptual Problems & Solutions

Redundancy and normalization tradeoff: The attributes venue_id are retained in both assignedToTier and blocks, even though a performance already determines its venue. This creates controlled redundancy and means the relations are not fully normalized under the identified functional dependency. However, venue_id is required to directly reference the composite keys of section and seat. Keeping it allows MySQL to enforce foreign-key constraints without relying on indirect joins, so the design prioritizes referential integrity and simpler validation over complete removal of redundancy
Representing reserved and general-admission sections: Initially, reserved and general sections were considered separate subtypes. This however creates unnecessary tables, so the final design stores a section_type attribute in section, while rows and seats exist only for reserved sections and capacity exists only for general admission sections
Preserving history while making seats available again: When a ticket is cancelled, its reserveSeat row is kept instead of deleted. Availability checks only treat active tickets as occupying seats, so the seat becomes available while the database still preserves which seat the cancelled ticket originally reserved.
Tracking changing ticket ownership: A ticket may be resold multiple times, so storing only a current owner would lose history. The ownsTicket relation records acquisition and end times, allowing one current owner while preserving all previous owners.
Price changes after ticket sales: Since organizers may later change tier prices, storing only the current tier price would lose the original purchase price. Therefore, ticket.face_value stores the price paid when the ticket was created.

2. Assumptions

- A user is either a customer or an organizer, they cannot be both
- Email addresses uniquely identify users.
- Credit card numbers are assumed to uniquely identify payment records
- A ticket cannot have more than one active resale listing at a time.
- Every event is managed by only one organizer
- Every event must have at least one performance
- Every performance belongs to exactly one event and takes place at exactly one venue.
- Every section is either reserved seating or general admission, but not both
- Reserved sections contain rows and seats, while general admission sections contain only a standing capacity
- Sections, rows, seats, genres, and price tiers are modeled as weak entities
- Each performance defines its own price tiers
- Each performance–section combination is assigned exactly one price tier
- Payment information is recorded with orders rather than customer accounts
- A customer may place multiple orders, but each order is for exactly one performance.
    - I.e. If a customer purchases tickets for multiple performances, separate orders must be created.
- A ticket belongs to exactly one order.
- General admission tickets do not reserve individual seats.
    - Reserved seating tickets reference a specific seat through the ReserveSeat relationship, while general admission tickets are associated only with a section.
- A ticket has exactly one current owner at any point in time.
- A resale listing may only be purchased once.
    - Once purchased, a listing is considered closed and cannot be purchased again.
- Reviews are associated with performances rather than directly with events or venues.
    - Since both the event and venue can be determined from a performance, a review references the attended performance and stores both the event rating and venue rating.
- Ticket face value is stored at the time of purchase.
    - This preserves the original purchase price even if organizers later modify price tiers for future ticket sales.
- A performance cannot have two priceTiers with the same tiername.
- A ticket can be cancelled only once by the customer.
- Each customer can review a performance they attended at most once (based on project).
- No two performances can take place at the same venue on the same date and at the same time.
- An event cannot have two performances occurring on the same date and at the same time.

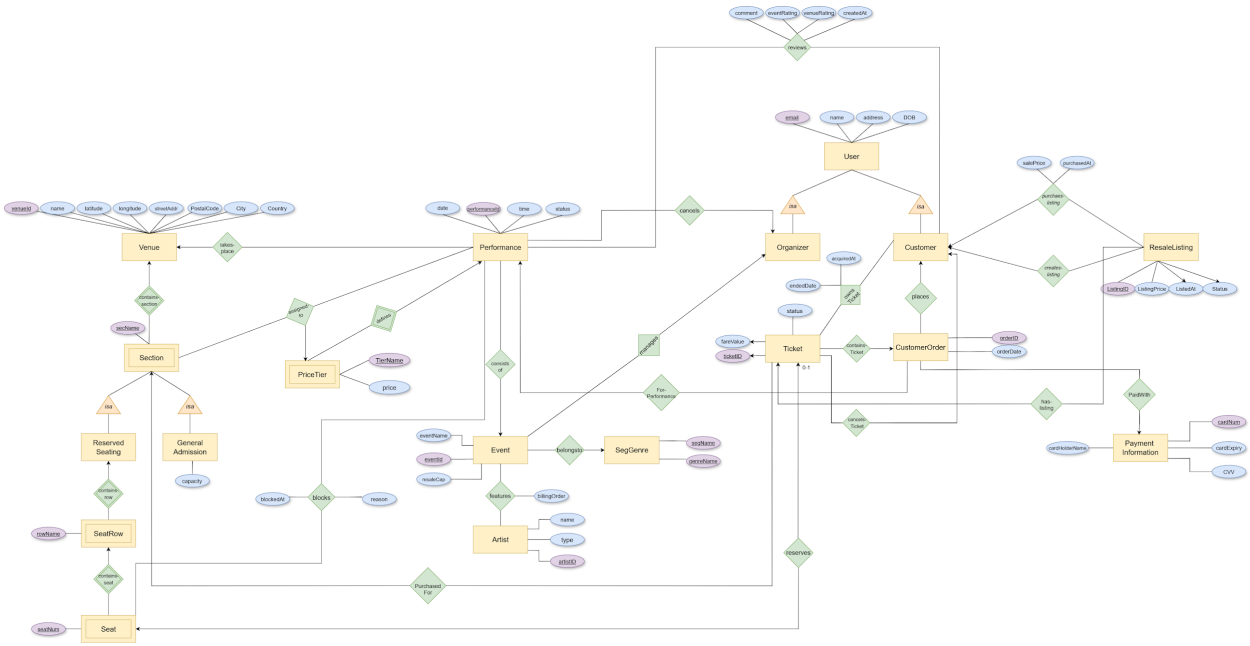
- Segment name and genre name together are considered unique since one genre can appear under a different segment (e.g classical appears under both concerts & Arts, Theatre & Comedy).
    - The same taxonomy as Ticketmaster was used, however only a select few were chosen for the purpose of this application
- A credit card number should be represented as 16 digit strings with a 3 digit CVV.

3. Design Choices

- If an organizer cancels a performance, each current ticket owner receives a refund equal to the ticket's original face value, regardless of any resale price.
- Each order contains tickets for one performance and one section. Reserved-seat and general-admission tickets must be purchased in separate orders.
- For query 3, finding performances in a specified range, we made a design choice to return only upcoming performances that have at least that required amount of tickets available.
- For query 1, we used haversine formula as a design choice to convert latitude/longitude to distance.
- For report 5 sub-report, we made an assumption that the question asks to rank those customers that have placed at least two orders in the past year, as recent customer purchasing activity is more relevant for evaluating customer behavior.
- For query 5, we assume users are mostly interested in knowing the upcoming performances, therefore for any filters applied, the results return upcoming performances (Scheduled) that match the specified filters.

Justification/Solution for Organizer Toolkit: The toolkit recommends prices by averaging recent performances of the same genre in the same city. If insufficient historical data exists, it falls back to a default three-tier pricing model (Premium, Standard, General) with a 25%/50%/25% capacity split.

4. ER Diagram



5. Relational Schema & Normalization

5.1. Entities

1. User(<u>email</u>, name, address, dateOfBirth)
2. Customer(<u>email</u>); email is a FK of User
3. Organizer(<u>email</u>); email is a FK of User
4. Event(<u>eventID</u>, organizerEmail, eventName, resaleCap)
    a. FK organizerEmail → Organizer.email
5. SegGenre(<u>segName</u>, <u>genreName</u>)
6. Artist(<u>artistID</u>, artistName, artistType)
7. Performance(<u>performanceID</u>, performanceDate, performanceTime, status, eventID, venueID)
    a. FK eventID → Event.eventID
    b. FK venueID → Venue.venueID
8. PriceTier(<u>performanceID</u>, <u>tierName</u>, price); performanceID is a FK of Performance
9. Venue(<u>venueID</u>, venueName, latitude, longitude, streetAddress, postalCode, city, country)
10. Section(<u>venueID</u>, <u>sectionName</u>, sectionType); venueID is a FK of Venue
11. GeneralAdmissionSection(<u>venueID</u>, <u>sectionName</u>, capacity);

    a. venueID & sectionName is a FK of Section
12. ReservedSection(venueID, sectionName); *(possibly remove this since already covered in section)*
    a. FK (venueID, sectionName) → Section
13. SeatRow(venueID, sectionName, rowName);
    a. FK (venueID, sectionName) → ReservedSection
14. Seat(venueID, sectionName, rowName, seatNum);
    a. FK (venueID, sectionName, rowName) → SeatRow
15. Ticket(ticketID, faceValue, status, orderID, venueID, sectionName)
    a. FK orderID → CustomerOrder.orderID
    b. FK (venueID, sectionName) → Section(venueID, sectionName)
16. CustomerOrder(orderID, orderDate, cardNum, performanceID, customerEmail)
    a. FK cardNum → PaymentInformation.cardNum
    b. FK performanceID → Performance.performanceID
    c. FK customerEmail → Customer.email
17. ResaleListing(listingID, listingPrice, listedAt, status, ticketID, sellerEmail)
    a. FK ticketID → Ticket.ticketID
    b. FK sellerEmail → Customer.email
18. PaymentInformation(cardNum, cardholderName, expiryDate, CVV)

5.2. Relationships

1. Features(eventID, artistID, billingOrder)
    a. FK eventID → Event.eventID
    b. FK artistID → Artist.artistID
2. BelongsTo(eventID, segName, genreName);
    a. FK eventID → Event.eventID
    b. FK (segName, genreName) → SegGenre
3. AssignedToTier(performanceID, venueID, sectionName, tierName)
    a. FK performanceID → Performance.performanceID
    b. FK (venueID, sectionName) → Section
    c. FK (performanceID, tierName) → PriceTier
4. Blocks(performanceID, venueID, sectionName, rowName, seatNum, blockedAt, reason)
    a. FK performanceID → Performance.performanceID
    b. FK (venueID, sectionName, rowName, seatNum) → Seat
5. ReserveSeat(ticketID, venueID, sectionName, rowName, seatNum)
    a. FK ticketID → Ticket.ticketID
    b. FK (venueID, sectionName, rowName, seatNum) → Seat
6. PurchaseListing(listingID, buyerEmail, purchasedAt, salePrice)
    a. FK listingID → ResaleListing.listingID

   b. FK buyerEmail → Customer.email
7. OwnsTicket(email, ticketID, acquiredAt, endedAt)
   a. FK email → Customer.email
   b. FK ticketID → Ticket.ticketID
8. Reviews(email, performanceID, eventRating, venueRating, comment, createdAt)
   a. FK email → Customer.email
   b. FK performanceID → Performance.performanceID
9. CancelsTicket(customerEmail, ticketID, cancelledAt, refundAmount, reason)
   a. FK customerEmail → Customer.email
   b. FK ticketID → Ticket.ticketID
10. CancelsPerformance(organizerEmail, performanceID, cancelledAt, reason)
   a. FK organizerEmail → Organizer.email
   b. FK performanceID → Performance.performanceID

5.3. FD's & Normalization

1. User(email, name, address, dob)
   Possible FD's:
   a. email -> name, address, dateOfBirth
   - Since email is the minimal superkey (based on our assumption that email addresses uniquely identify users), it determines all other attributes on the RHS. Therefore, the relation is in BCNF/3NF.
2. Customer(email)
   No nontrivial FD's
   a. Explanation: Since the Customer relation has no non-key attributes and has only trivial functional dependencies, the relation is in BCNF/3NF.
3. Organizer(email)
   No nontrivial FD's
   a. Explanation: Since the Organizer relation has no non-key attributes and has only trivial functional dependencies, the relation is in BCNF/3NF.
4. Event(eventID, organizerEmail, eventName, resaleCap)
   Possible FD's:
   a. eventID -> organizerEmail, eventName, resaleCap
   - Explanation: Since eventID is a minimal superkey that uniquely identifies each event and determines all attributes in the Event relation, the relation is in BCNF/3NF.
5. SegGenre(segName, genreName)
   No nontrivial FD's

    a. Explanation: Since the SegGenre relation has no non-key attributes and has only trivial functional dependencies, it is in BCNF/3NF.

6. Artist(<u>artistID</u>, artistName, artistType)
   Possible FD's:
    a. artistID -> artistName, artistType
    - Explanation: Since artistID is a minimal superkey that is unique to each artist and determines all attributes in the Artist relation, the relation is in BCNF/3NF.
7. Performance(<u>performanceID</u>, performanceDate, performanceTime, status, eventID, venueID)
   Possible FD's:
    a. performanceID -> performanceDate, performanceTime, status, eventID, venueID
    b. performanceDate, performanceTime, venueID -> status, eventID, performanceID
    c. performanceDate, performanceTime, eventID -> status, venueID, performanceID
    - Explanation: For all FD's (a, b, c), attributes on the LHS are superkeys that uniquely determine all the attributes in the Performance relation, therefore, the relation is in BCNF/3NF.
8. PriceTier(<u>performanceID</u>, <u>tierName</u>, price)
   Possible FD's:
    a. performanceID, tierName -> price
    - Explanation: Since performanceID, and tierName on the LHS together form a minimal superkey that uniquely determines all the attributes in the PriceTier relation, the relation is in BCNF/3NF.
9. Venue(<u>venueID</u>, venueName, latitude, longitude, streetAddress, postalCode, city, country)
   Possible FD's:
    a. venueID -> venueName, latitude, longitude, streetAddress, postalCode, city, country
    - Explanation: Since venueID is a minimal superkey that uniquely identifies each venue and determines all attributes in the Venue relation, the relation is in BCNF/3NF.
10. Section(<u>venueID</u>, <u>sectionName</u>, sectionType)
    Possible FD's:
    a. venueID, sectionName ->sectionType
    - Explanation: Since venueID, and sectionName on the LHS together form a minimal superkey that uniquely determines all the attributes in the Section relation, the relation is in BCNF/3NF.
11. GeneralAdmissionSection(<u>venueID</u>, <u>sectionName</u>, capacity)
    Possible FD's:
    a. venueID, sectionName -> capacity

- Explanation: Since venueID, and sectionName on the LHS together form a minimal superkey that uniquely determines all the attributes in the GeneralAdmissionSection relation, the relation is in BCNF/3NF.

12. SeatRow(<u>venueID</u>, <u>sectionName</u>, <u>rowName</u>)
    No nontrivial FD's
    a. Explanation: Since the SeatRow relation has no non-key attributes and has only trivial functional dependencies, the relation is in BCNF/3NF.
13. Seat(<u>venueID</u>, <u>sectionName</u>, <u>rowName</u>, <u>seatNum</u>)
    No nontrivial FD's
    a. Explanation: Since the Seat relation has no non-key attributes and has only trivial functional dependencies, the relation is in BCNF/3NF.
14. Ticket(<u>ticketID</u>, faceValue, status, orderID, venueID, sectionName)
    Possible FD's:
    a. ticketID -> faceValue, status, orderID, venueID, sectionName
    b. orderID -> performanceID -> venueID => orderID -> venueID
    - Explanation: Although orderID is redundant (orderID -> venueID) because orderID is not a superkey, we are keeping the original Ticket relation as a design decision because sectionName in the "Section" table is only unique within a specific venue and venueID is required to reference the section table using the foreignkey constraint.
15. CustomerOrder(<u>orderID</u>, orderDate, cardNum, performanceID, customerEmail)
    Possible FD's:
    a. orderID -> orderDate, cardNum, performanceID, customerEmail
    - Explanation: Since OrderID is a minimal superkey that uniquely identifies each order and determines all attributes in the CustomerOrder relation, the relation is in BCNF/3NF.
16. ResaleListing(<u>listingID</u>, listingPrice, listedAt, status, ticketID, sellerEmail)
    Possible FD's:
    a. listingID -> listingPrice, listedAt, status, ticketID, sellerEmail
    - Explanation: Since listingID is a minimal superkey that uniquely identifies each listing and determines all attributes in the ResaleListing relation, the relation is in BCNF/3NF.
17. PaymentInformation(<u>cardNum</u>, cardholderName, expiryDate, CVV)
    a. cardNum -> cardholderName, expiryDate, CVV
    - Explanation: Since CardNum is a minimal superkey that is used to identify payment information (based on our assumption) and determines all attributes in the PaymentInformation relation, the relation is in BCNF/3NF.

18. Features(<u>eventID</u>, <u>artistID</u>, billingOrder)
    Possible FD's:

a. eventID, artistID -> billingOrder
- Explanation: Since an event features one or more artists, eventID together with the artistID uniquely determines the billingOrder, the relation is in BCNF/3NF.

19. BelongsTo(eventID, segName, genreName);
Possible FD's:
a. eventID -> segName, genreName
- Explanation: Since each event belongs to exactly one segment and genre pair, and the eventID is a minimal superkey, the relation is in BCNF/3NF.

20. AssignedToTier(performanceID, sectionName, venueID, tierName)
Possible FD's:
a. performanceID, sectionName -> tierName
b. performanceID -> venueID *(violates BCNF, since performanceID is not the superkey)*
c. performanceID, sectionName -> venueID
- performanceID, sectionName together forms a minimal superkey (primary key)
- **Normalization:**
  - performanceID+ = performanceID, venueID
  - R1 (performanceID, venueID), FD's: performanceID -> venueID
  - R2 (performanceID, sectionName, tierName), FD's: performanceID, sectionName -> tierName
- Although venueID is redundant (performanceID -> venueID) because performanceID is not a superkey, we are keeping the original AssignedToTier relation as a design decision because sectionName in the "Section" table is only unique within a specific venue and venueID is required to reference the section table using the foreignkey constraint.

21. Blocks(performanceID, sectionName, rowName, seatNum, venueID, blockedAt, reason)
Possible FD's:
a. performanceID -> venueID *(violates BCNF, since performanceID is not the superkey)*
b. performanceID, sectionName, rowName, seatNum -> blockedAt, reason
- **Normalization:**
  - performanceID+ = performanceID, venueID
  - R3 (performanceID, venueID), FD's: performanceID -> venueID
  - R4 (performanceID, sectionName, rowName, seatNum, blockedAt, reason), FD's: performanceID, sectionName, rowName, seatNum -> blockedAt, reason
- Although venueID is redundant (performanceID -> venueID) because performanceID is not a superkey, we are keeping the original AssignedToTier relation as a design decision because sectionName in the "Section" table is only

unique within a specific venue and venueID is required to reference the section table using the foreignkey constraint.

22. ReserveSeat(<u>ticketID</u>, venueID, sectionName, rowName, seatNum)
    Possible FD's:
    a. ticketID -> sectionName, venueID, rowName, seatNum
    - Explanation: Since ticketID is a minimal superkey that uniquely identifies exactly one seat in a particular row and section for that venue, the relation is in BCNF/3NF.
23. PurchaseListing(<u>listingID</u>, buyerEmail, purchasedAt, salePrice)
    Possible FD's:
    a. listingID -> buyerEmail, purchasedAt, salePrice
    - Explanation: Since listingID is a minimal superkey and is associated with exactly one buyer email, purchase date, and sale Price, the relation is in BCNF/3NF.
24. OwnsTicket(<u>ticketID</u>, <u>acquiredAt</u>, email, endedAt)
    Possible FD's:
    a. ticketID, acquiredAt -> email, endedAt
    - Explanation: Since a particular ticket can be associated with many owners throughout its history, the ticketID and the date/time when the ownership of that ticket was acquired uniquely identify each tuple, and determine all the attributes of the OwnsTicket relation. Based on our assumption, since exactly one customer can be the owner of a ticket during a particular time period, the relation is in BCNF/3NF.
25. Reviews(<u>email</u>, <u>performanceID</u>, eventRating, venueRating, comment, createdAt)
    Possible FD's:
    a. email, performanceID -> eventRating, venueRating, comment, createdAt
    - Explanation: Since, the combination of email and performanceID is a minimal super key that identifies all the attributes in the Reviews relation where each customer can review a performance they attended at most once, the relation is in BCNF/3NF.
26. CancelsTicket(<u>ticketID</u>, customerEmail, cancelledAt, refundAmount, reason)
    Possible FD's:
    a. ticketID -> customerEmail, cancelledAt, refundAmount, reason
    - Explanation: Since, ticketID is a minimal superkey and each ticket can be cancelled only once based on our assumption, the relation is in BCNF/3NF.
27. CancelsPerformance(<u>performanceID</u>, organizerEmail, cancelledAt, reason)
    Possible FD's:
    a. performanceID -> organizerEmail, cancelledAt, reason
    - Explanation: Since, performanceID is a minimal superkey and each performance is owned by one organizer, the relation is in BCNF/3NF.

5.4. DDL statements for schema design

1. CREATE TABLE users (
   email VARCHAR(255) PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   dob DATE NOT NULL,
   address VARCHAR(500) NOT NULL
)

2. CREATE TABLE customer (
   email VARCHAR(255) PRIMARY KEY,
   FOREIGN KEY (email) REFERENCES users(email)
   ON DELETE CASCADE
)

3. CREATE TABLE organizer (
   email VARCHAR(255) PRIMARY KEY,
   FOREIGN KEY (email) REFERENCES users(email)
   ON DELETE CASCADE
)
4. CREATE TABLE events (
   event_id INT AUTO_INCREMENT PRIMARY KEY,
   organizer_email VARCHAR(255) NOT NULL,
   event_name VARCHAR(100) NOT NULL,
   resale_cap DECIMAL(4, 2) NOT NULL,
   FOREIGN KEY (organizer_email) REFERENCES organizer(email)
)

5. CREATE TABLE segGenre (
   seg_name VARCHAR(100) NOT NULL,
   genre_name VARCHAR(100) NOT NULL,
   PRIMARY KEY (seg_name, genre_name)
)

6. CREATE TABLE artist (
   artist_id INT AUTO_INCREMENT PRIMARY KEY,
   artist_name VARCHAR(100) NOT NULL,
   artist_type VARCHAR(10) NOT NULL,

```
  CHECK (artist_type = 'artist' or artist_type = 'team')
);
```

7.
```
CREATE TABLE venue (
  venue_id     INT   AUTO_INCREMENT PRIMARY KEY,
  venue_name   VARCHAR(100)    NOT NULL,
  latitude   DECIMAL(9, 6)   NOT NULL,
  longitude      DECIMAL(9, 6)   NOT NULL,
  street_address    VARCHAR(255)    NOT NULL,
  postal_code       VARCHAR(10)     NOT NULL,
  city         VARCHAR(100)    NOT NULL,
  country   VARCHAR(100)    NOT NULL
)
```

8.
```
CREATE TABLE performance (
  performance_id     INT    PRIMARY KEY,
  performance_date       DATE    NOT NULL,
  performance_time       TIME     NOT NULL,
  performance_status     VARCHAR(50)   NOT NULL,
  event_id     INT       NOT NULL,
  venue_id    INT       NOT NULL,
  FOREIGN KEY (event_id) REFERENCES events(event_id),
  FOREIGN KEY (venue_id) REFERENCES venue(venue_id),
  CHECK (performance_status = 'scheduled' or performance_status = 'cancelled' or
performance_status = 'completed'),
  UNIQUE (venue_id, performance_date, performance_time),
  UNIQUE (event_id, performance_date, performance_time)
)
```

9.
```
CREATE TABLE priceTier (
  performance_id      INT     NOT NULL,
  tier_name     VARCHAR(10)     NOT NULL,
  price             DECIMAL(6, 2)   NOT NULL,
  PRIMARY KEY (performance_id, tier_name),
  FOREIGN KEY (performance_id) REFERENCES performance(performance_id)
)
```

10.
```
CREATE TABLE section (
  venue_id       INT     NOT NULL,
  section_name    VARCHAR(100)    NOT NULL,
```

section_type VARCHAR(10) NOT NULL,
PRIMARY KEY (venue_id, section_name),
FOREIGN KEY (venue_id) REFERENCES venue(venue_id),
CHECK (section_type = 'general' or section_type = 'reserved')
)

11. CREATE TABLE generalAdmissionSection (
venue_id INT NOT NULL,
section_name VARCHAR(100) NOT NULL,
capacity INT NOT NULL,
PRIMARY KEY (venue_id, section_name),
FOREIGN KEY (venue_id, section_name) REFERENCES section(venue_id, section_name)
)

12. CREATE TABLE seatRow (
venue_id INT NOT NULL,
section_name VARCHAR(100) NOT NULL,
row_name VARCHAR(100) NOT NULL,
PRIMARY KEY (venue_id, section_name, row_name),
FOREIGN KEY (venue_id, section_name) REFERENCES section(venue_id, section_name)
)

13. CREATE TABLE seat (
venue_id INT NOT NULL,
section_name VARCHAR(100) NOT NULL,
row_name VARCHAR(100) NOT NULL,
seat_num INT NOT NULL,
PRIMARY KEY (venue_id, section_name, row_name, seat_num),
FOREIGN KEY (venue_id, section_name, row_name) REFERENCES seatRow(venue_id, section_name, row_name)
)

14. CREATE TABLE paymentInformation (
card_num VARCHAR(16) PRIMARY KEY,
cardholder_name VARCHAR(100) NOT NULL,
card_expiry DATE NOT NULL,
cvv VARCHAR(3) NOT NULL
)

15. CREATE TABLE customerOrder (
order_id INT PRIMARY KEY,
order_date DATETIME NOT NULL DEFAULT (NOW()),
card_num VARCHAR(16) NOT NULL,
performance_id INT NOT NULL,
customer_email VARCHAR(255) NOT NULL,
FOREIGN KEY (performance_id) REFERENCES performance(performance_id),
FOREIGN KEY (customer_email) REFERENCES customer(email),
FOREIGN KEY (card_num) REFERENCES paymentInformation(card_num)
)

16. CREATE TABLE ticket (
ticket_id INT PRIMARY KEY,
face_value DECIMAL(6, 2) NOT NULL,
ticket_status VARCHAR(50) NOT NULL,
order_id INT NOT NULL,
venue_id INT NOT NULL,
section_name VARCHAR(100) NOT NULL,
FOREIGN KEY (venue_id, section_name) REFERENCES section(venue_id, section_name),
FOREIGN KEY (order_id) REFERENCES customerOrder(order_id),
CHECK (ticket_status = 'active' or ticket_status = 'cancelled' or ticket_status = 'refunded')
)

17. CREATE TABLE resaleListing (
listing_id INT PRIMARY KEY,
listing_price DECIMAL(6,2) NOT NULL,
listed_at DATETIME NOT NULL DEFAULT (NOW()),
listing_status VARCHAR(50) NOT NULL,
ticket_id INT NOT NULL,
seller_email VARCHAR(255) NOT NULL,
FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id),
FOREIGN KEY (seller_email) REFERENCES customer(email),
CHECK (listing_status = 'active' or listing_status = 'withdrawn' or listing_status = 'sold')
)

18. CREATE TABLE features (
event_id INT NOT NULL,

```
  artist_id      INT    NOT NULL,
  billing_order        VARCHAR(100)    NOT NULL,
  PRIMARY KEY (event_id, artist_id),
  FOREIGN KEY (event_id) REFERENCES events(event_id),
  FOREIGN KEY (artist_id) REFERENCES artist(artist_id),
  CHECK (billing_order = 'Headliner' or billing_order = 'Special Guest' or billing_order =
'Opening Act')
  )
```

19.
```
CREATE TABLE belongsTo (
  event_id       INT      NOT NULL,
  seg_name     VARCHAR(100)    NOT NULL,
  genre_name  VARCHAR(100)    NOT NULL,
  PRIMARY KEY (event_id),
  FOREIGN KEY (event_id) REFERENCES events(event_id),
  FOREIGN KEY (seg_name, genre_name) REFERENCES segGenre(seg_name,
genre_name)
  )
```

20.
```
CREATE TABLE assignedToTier (
  performance_id    INT       NOT NULL,
  venue_id             INT       NOT NULL,
  section_name       VARCHAR(100)    NOT NULL,
  tier_name            VARCHAR(10)     NOT NULL,
  PRIMARY KEY (performance_id, section_name),
  FOREIGN KEY (performance_id, tier_name) REFERENCES
priceTier(performance_id, tier_name),
  FOREIGN KEY (venue_id, section_name) REFERENCES section(venue_id,
section_name)
  )
```

21.
```
CREATE TABLE blocks (
  performance_id     INT      NOT NULL,
  section_name       VARCHAR(100)    NOT NULL,
  venue_id             INT           NOT NULL,
  row_name           VARCHAR(100)    NOT NULL,
  seat_num           INT           NOT NULL,
  blocked_at         DATETIME       NOT NULL DEFAULT (NOW()),
  reason               VARCHAR(500)    NOT NULL,
  PRIMARY KEY (performance_id, section_name, row_name, seat_num),
```

```
  FOREIGN KEY (performance_id) REFERENCES performance(performance_id),
  FOREIGN KEY (venue_id, section_name, row_name, seat_num) REFERENCES
seat(venue_id, section_name, row_name, seat_num)
)
```

22.
```
CREATE TABLE reserveSeat (
  ticket_id       INT          NOT NULL,
  section_name    VARCHAR(100)    NOT NULL,
  venue_id        INT      NOT NULL,
  row_name        VARCHAR(100)    NOT NULL,
  seat_num        INT          NOT NULL,
  PRIMARY KEY (ticket_id),
  FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id),
  FOREIGN KEY (venue_id, section_name, row_name, seat_num) REFERENCES
seat(venue_id, section_name, row_name, seat_num)
)
```

23.
```
CREATE TABLE purchaseListing (
  listing_id       INT          NOT NULL,
  buyer_email   VARCHAR(255)    NOT NULL,
  purchased_at DATETIME       NOT NULL DEFAULT (NOW()),
  sale_price       DECIMAL(6, 2)   NOT NULL,
  PRIMARY KEY (listing_id),
  FOREIGN KEY (listing_id) REFERENCES resaleListing(listing_id),
  FOREIGN KEY (buyer_email) REFERENCES customer(email)
)
```

24.
```
CREATE TABLE ownsTicket (
  ticket_id    INT          NOT NULL,
  acquired_at    DATETIME       NOT NULL DEFAULT (NOW()),
  email          VARCHAR(255)    NOT NULL,
  ended_at       DATETIME       DEFAULT NULL,
  PRIMARY KEY (ticket_id, acquired_at),
  FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id),
  FOREIGN KEY (email) REFERENCES customer(email)
)
```

25.
```
CREATE TABLE reviews (
  email              VARCHAR(255)    NOT NULL,
  performance_id     INT          NOT NULL,
```

```
    event_rating        INT          NOT NULL,
    venue_rating        INT          NOT NULL,
    comment             VARCHAR(1000)   NOT NULL,
    created_at          DATETIME       NOT NULL DEFAULT (NOW()),
    PRIMARY KEY (email, performance_id),
    FOREIGN KEY (performance_id) REFERENCES performance(performance_id),
    FOREIGN KEY (email) REFERENCES customer(email),
    CHECK (event_rating >= 1 and event_rating <= 5),
    CHECK (venue_rating >= 1 and venue_rating <= 5)
  )
```

26.
```
CREATE TABLE cancelsTicket (
    ticket_id       INT      NOT NULL,
    customer_email   VARCHAR(255)    NOT NULL,
    refund_amount    DECIMAL(6, 2)   NOT NULL,
    reason              VARCHAR(500)    NOT NULL,
    cancelled_at       DATETIME       NOT NULL DEFAULT (NOW()),
    PRIMARY KEY (ticket_id),
    FOREIGN KEY (ticket_id) REFERENCES ticket(ticket_id),
    FOREIGN KEY (customer_email) REFERENCES customer(email),
    CHECK (refund_amount >= 0)
  )
```

27.
```
CREATE TABLE cancelsPerformance (
    performance_id      INT          NOT NULL,
    organizer_email     VARCHAR(255)    NOT NULL,
    reason           VARCHAR(500)    NOT NULL,
    cancelled_at      DATETIME    NOT NULL DEFAULT (NOW()),
    PRIMARY KEY (performance_id),
    FOREIGN KEY (performance_id) REFERENCES performance(performance_id),
    FOREIGN KEY (organizer_email) REFERENCES organizer(email)
  )
```

6. References

https://dev.mysql.com/doc/refman/8.0/en/
https://dev.mysql.com/doc/refman/8.0/en/window-functions-usage.html
https://docs.oracle.com/javase/8/docs/api/java/sql/package-summary.html